

Guion de sustentación — Primer Corte · Restricción RES-05

Formato: 12 min de exposición + 8 min de preguntas. 2 slides. Equipo de 4. **Regla del equipo:** los 4 dominan el discurso completo; cada bloque tiene un conductor asignado, pero cualquiera debe estar listo para responder el turno de preguntas. **Demo en vivo:** obligatoria. Ejecución real de los 3 comandos sobre el tag `corte-1` (50b92f8).

1. Reparto por tiempo (12 min)

Min	Bloque	Conductor	Contenido mínimo
0–2	Contexto + restricción	Miembro A	Producto (Click & Collect, 5 tiendas), qué exige RES-05 y por qué es crítico (ESC-02)
2–5	Slide 1: diagnóstico y decisión	Miembros B–C	Línea base 2/2, alternativas A/B/C, por qué B, trade-offs
5–9	Slide 2: corte vertical en vivo	Miembro D	Explicar flujo + ejecutar los 3 comandos
9–12	Verificación, trazabilidad y crítica	Miembro A (cierra)	0/300, 13/13, cadena RF-02→ESC-02→A-02→ADR-0004; límites honestos

Antes de subir: ejecutar `npm test` y `node scripts/medir-aislamiento.js` y tener el terminal listo.

2. Definiciones clave · contexto oral (cualquiera debe poder explicarlas)

Concepto	Qué explicar
Corte vertical	Tajada ejecutable que atraviesa la arquitectura de punta a punta con lógica de negocio real: <code>catalogo</code> → <code>pedidos</code> → <code>pagos</code> → <code>entrega</code> → <code>notificaciones</code> . Prueba el valor completo de un pedido sin UI. Vive en <code>src/corte-vertical.js</code> con prueba automatizada. No es una "demo de pantallas": es el dominio funcionando.
Pipeline (CI)	<code>.github/workflows/ci.yml</code> . Cada push/PR a <code>master</code> dispara: <code>job test</code> (<code>npm ci</code> → <code>npm test</code>) y <code>job sonar</code> (análisis estático). Un fallo en un job deja el commit en rojo → "rompe el CI". La CI en verde es un criterio de la rúbrica (etiqueta <code>corte-1</code> sobre un commit con CI verde, run <code>34064927441</code>).
Pipeline ≠ SonarCloud	El pipeline es la tubería completa (test + sonar). SonarCloud es solo uno de sus pasos y está configurado pero inactivo (sin <code>SONAR_TOKEN</code> ni org/projectKey reales).
Cómo interpretar las pruebas	<code>npm test</code> usa el runner nativo <code>node --test</code> (Node ≥ 22, ESM). Reporta N tests pass / N fail a partir de 4 archivos: <code>health</code> , <code>modulos</code> , <code>corte-vertical</code> y <code>aislamiento</code> . 13/13 en verde = salud + reglas de dominio + flujo completo + 4 pruebas

Concepto	Qué explicar
	específicas de aislamiento. Que pasen todas no "prueba" nada más de lo que cubren: cubren comportamiento, no rendimiento ni seguridad.
Línea base vs. post-cambio	Misma herramienta, mismo procedimiento, dos estados del código: sobre 812d227 (pre) se midieron 2/2 fugas ; sobre corte-1 (post) 0/300 . Eso es "medición reproducible", no una afirmación del equipo.
Aislamiento estructural vs. convencional	<i>Estructural</i> : <code>Map<tiendaId, Map<...>></code> hace imposible tocar el estado de otra tienda; no depende de que el llamador "filtre bien". <i>Convencional</i> (alt. A): el estado es global y la protección depende del uso correcto — por eso se descartó.
tiendald	Parámetro obligatorio en las 5 firmas de módulo; si falta → error. Da contexto de enrutamiento y permite que pedido-1 exista en varias tiendas sin colisionar.
Máquina de estados	Recibido → En preparación → Listo → Entregado . Solo transiciones secuenciales (ESTADOS en <code>src/modules/pedidos/index.js</code>); un salto se rechaza.
PIN	Código de 4 dígitos generado en <code>marcarListo</code> ; se valida en <code>validarPin</code> (solo en estado Listo). Es el único control del punto de recolección (ESC-03 / A-06; el QR se descartó).
arc42 · C4 · ADR	arc42 = plantilla de documentación (secciones 1–12, con la §11 dedicada al reto RES-05). C4 = nivel de abstracción: contexto y contenedores (<code>docs/c4/</code>). ADR = registro de cada decisión con alternativas, trade-offs y trazabilidad (<code>docs/adr/</code>). Nuestro flujo: ADR-0001 (monolito modular) → 0002 (ratificación) → 0003 (Railway/Docker/SonarCloud) → 0004 (RES-05).
Monolito modular	Un solo proceso desplegable. Los 5 módulos (<code>src/modules/*</code>) tienen frontera estricta: comunicación solo por interfaz pública (<code>index.js</code>), nunca acceso directo a la capa de datos de otro módulo. Esto es lo que permite migrar después (PostgreSQL, o extraer un módulo) sin reescribir.
Trade-off / costo de reversión	Toda decisión arquitectónica tiene costo. Aquí anticipamos dos rutas de salida: el umbral del 40 % (opción C) y la migración a PostgreSQL ; ambas con costo de reversión bajo porque el aislamiento por tienda no cambia interfaces.
RES-05	Restricción asignada al equipo en corte 1: operar siempre en contexto de tienda y prohibido tocar estado de otra tienda. Umbral de ESC-02: 0 accesos cruzados .
Quality Gate / SonarCloud	Análisis estático (bugs, vulnerabilidades, duplicación, cobertura). Decidido en ADR-0003; configurado en <code>sonar-project.properties</code> y en el CI, pero pendiente de org/projectKey reales + <code>SONAR_TOKEN</code> .

3. Narrativa por bloque

Bloque A · 0–2 min — Contexto y restricción (Miembro A)

Idea central: LaPlacita agrupa **5 tiendas independientes** en una sola plataforma; que una tienda lea o mute lo de otra es un error operativo grave (enrutar mal un pedido, ver el inventario ajeno). Eso es exactamente la

restricción **RES-05** que se nos asignó.

Responder: qué es el producto (Click & Collect, pedir y recoger con PIN), quiénes lo usan, por qué la multi-tenencia es el riesgo central → escenario **ESC-02** (umbral: 0 registros de otra tienda) → requisito **RF-02** → aspecto **A-02**.

Puntos para tocar si el jurado pregunta aquí:

- ¿"Aislamiento" quiere decir qué exactamente? → que catálogo, pedidos y entrega solo operen dentro del `tiendaId` propio.
- ¿Por qué es de alta prioridad? → 5 negocios comparten plataforma; la desconfianza por datos mezclados rompe el modelo (A-02).

Bloque B · 2–3.5 min — Diagnóstico (Miembro B)

Idea central: el problema no era de uso, era **estructural**. Medimos el estado inicial (commit `812d227`) con `scripts/medir-aislamiento.js` y el resultado fue **2/2 accesos cruzados logrados**: cualquier llamador leía o mutaba pedidos de cualquier tienda.

Evidencia para citar:

- `obtenerPedido(pedidoId)` y `cambiarEstado(pedidoId)` operaban sobre un `Map` global, sin `tiendaId`.
- `obtenerProducto(productoId)` no verificaba la tienda del producto.
- `marcarListo` / `validarPin` operaban sin contexto de tienda.
- Resultado medido: **2/2 fugas** → incumplía el umbral de ESC-02 (0).

Responder si preguntan: ¿cómo se midió? → herramienta reproducible + carga (ver §5.2). ¿Por qué 2 y no más? → el script intenta leer pedido, mutar estado y leer producto de otra tienda; los tres eran alcanzables.

Bloque C · 3.5–5 min — Decisión y trade-offs (Miembro C)

Idea central: evaluamos 3 alternativas y documentamos todo en **ADR-0004**.

Alternativa	Veredicto	Razón
A. Filtrar sobre repositorio global	Descartada	El estado global persiste → la protección depende del llamador (aislamiento convencional)
B. Repositorio particionado por tienda (<code>Map<tiendaId,...></code>)	Elegida	Aislamiento estructural, sin infraestructura nueva (RES-02/RES-03), IDs no colisionan
C. Servicio y BD por tienda	Descartada	Aislamiento físico, pero multiplica despliegues → contradice equipo de 4 y un semestre

Trade-offs a defender:

- **Positivos:** el aislamiento lo impone la estructura, no la disciplina; 0 accesos cruzados verificable; los IDs coinciden entre tiendas sin colisionar; el monolito se despliega igual.
- **Negativos / costo:** hubo que cambiar las firmas de los 5 módulos y sus pruebas; una futura capa HTTP deberá acarrear `tiendaId` en todas las rutas (ya previsto en la interfaz).

- **Costo de reversión:** bajo — el repositorio por tienda se sustituye por PostgreSQL sin cambiar interfaces.
- **Disparador de revisión:** >40 % del tráfico en una tienda → alternativa C.

Bloque D · 5–9 min — Corte vertical en vivo (Miembro D)

Idea central: esto **no se lee, se ejecuta**. El corte vertical es la prueba viviente de que la respuesta a RES-05 funciona.

Narrativa antes de ejecutar (30 s): el pedido nace en **catalogo** (con **tiendaId**), pasa por **pedidos** (creación + máquina de estados), **pagos** (confirmación), **entrega** (PIN) y **notificaciones** (historial). Desde el corte-1, **toda operación exige **tiendaId****.

Secuencia exacta (no improvisar):

```
npm test                # → 13 tests pass / 0 fail
node scripts/medir-aislamiento.js  # → accesosCruzadosLogrados=0, cumple=true,
exit 0
node src/corte-vertical.js        # → 4 estados (Recibido→En
preparación→Listo→Entregado) + PIN + historial
```

Al explicar la salida:

- **npm test:** 13/13 — 4 archivos de pruebas (health, módulos, corte vertical, aislamiento).
- **medir-aislamiento.js:** 100 ciclos × 3 operaciones cruzadas = **300 intentos**; si cualquier fuga llegara a 1, **exit 1** y no cumple. Con 0 → umbral ESC-02 cumplido.
- **corte-vertical.js:** el flujo que acabamos de ver en el slide 2, con el PIN generado y el historial de notificaciones.

Cierre del bloque D (si queda tiempo): la medición es **reproducible**, no una declaración: mismo script, mismo procedimiento, dos estados del código (base 2/2 → post 0/300).

Bloque E · 9–12 min — Verificación, trazabilidad y crítica (Miembro A cierra)

Trazabilidad (cadena completa): RF-02 → A-02 → ESC-02 → ADR-0004 → código → tests → evidencia → docs/ia.md (06/09). Cada ADR enlaza su commit; arc42 §11 documenta el reto completo.

Crítica honesta (evaluar la propia arquitectura):

- La persistencia actual es **en memoria** (**Map**): correcta para este corte y para la medición, **no** es producción. El plan es PostgreSQL (Corte 2), y el ADR-0001 ya lo prevé.
- **SonarCloud configurado pero no activo:** faltan org/projectKey reales y **SONAR_TOKEN**. Está **declarado como pendiente**, no ocultado.
- Aún **no existe capa HTTP**: las rutas que deberán acarrear **tiendaId** están previstas en la interfaz del módulo, pero no implementadas.
- **Sin telemetría de tráfico por tienda:** el disparador del 40 % hoy no se puede detectar con métricas propias.

Cierre: la decisión B responde RES-05 con evidencia medible, mantiene las restricciones del semestre y deja trazadas dos rutas de evolución (PostgreSQL, extracción de tienda >40 %) sobre un diseño que no cambia interfaces.

4. Demo en vivo — checklist

- ☐ Terminal abierto en la raíz del repo, HEAD en `50b92f8` (tag `corte-1`).
- ☐ `npm test` → confirmar `13 pass, 0 fail`.
- ☐ `node scripts/medir-aislamiento.js` → confirmar `accesosCruzadosLogrados=0` y `cumple=true`.
- ☐ `node src/corte-vertical.js` → confirmar los 4 estados + PIN + historial de notificaciones.
- ☐ Si un comando falla en vivo: decir "la medición es reproducible, lo re-ejecutamos" y correr de nuevo; no inventar resultados.

5. Respuestas preparadas (preguntas probables del jurado)

5.1 ¿Por qué el commit que implementó RES-05 rompió el CI por SonarCloud, y qué le faltó a la configuración inicial?

Respuesta (verificada en el historial):

1. `95ec841` (implementación de RES-05) añadió el job `sonar` con la condición `if: ${ secrets.SONAR_TOKEN != '' }` **a nivel de job**.
2. **GitHub Actions no permite evaluar el contexto `secrets` en expresiones `if`** (ni de job ni de paso). La expresión se evalúa sin el valor del secreto → el comportamiento del job no era el esperado.
3. Intentos de corrección: `b097955` movió la condición al paso (aún con `secrets`, seguía mal); `50b92f8` lo resolvió exponiendo `SONAR_TOKEN` en el `env` del job y evaluando `if: ${ env.SONAR_TOKEN != '' }` en el paso.
4. **Lo que le faltó a la configuración inicial, además de la sintaxis:**
 - `sonar-project.properties` con valores **placeholder** (`sonar.organization=REEMPLAZAR-POR-ORGANIZACION-SONARCLOUD`, `sonar.projectKey=REEMPLAZAR-POR-PROJECTKEY-SONARCLOUD`): aunque hubiera token, el análisis fallaría sin una org/proyecto reales.
 - El secreto `SONAR_TOKEN` **no está configurado** en el repositorio: por eso el paso se auto-omite.
 - El job `sonar` duplicaba `npm ci` + `npm test` (ineficiente pero no un fallo).
5. Estado actual honesto: **pipeline en verde, análisis SonarCloud pendiente de activación** (declarado en `entregable-corte1.md` §5 y en `sonar-project.properties`).

Si preguntan "¿entonces el CI está roto?" → No: el job de pruebas corre y está verde (`run 34064927441`); lo que está omitido es el análisis de SonarCloud porque sin token/org no podría correr.

5.2 ¿Qué pasaría si una tienda concentrara más del 40 % del tráfico? ¿Ya hay un plan concreto?

Respuesta (dos partes: criterio + plan real):

1. **El criterio está definido desde ADR-0001 §4** (Consecuencias) y citado en ADR-0004 (§Riesgos): si una tienda concentra **>40 % del tráfico**, se evalúa **extraer solo el módulo de esa tienda** (o el

procesamiento de pedidos) hacia **un microservicio independiente** para balancear la carga.

2. **Plan concreto: no está implementado.** La crítica honesta:

- **Falta telemetría:** hoy la persistencia es en memoria y no medimos tráfico por tienda; no hay indicadores que disparen el umbral.
- No hay runbook ni ADR de ejecución del escenario.

3. **Por qué la decisión de hoy facilita esa ruta:** el aislamiento por tienda del ADR-0004

(`Map<tiendaId,...>` con ids por tienda) deja los datos de cada tienda **separados por construcción**.

Extraer una tienda a su propio servicio **no cambiaría interfaces** → costo de reversión bajo (así lo documenta el ADR-0004).

4. **Plan que propondríamos si se disparara (respuesta de diseño):**

- Corte 2 habilita la observabilidad con los contenedores ya planeados (PostgreSQL + Redis) → medir pedidos/tienda/hora.
- Superado el umbral de manera sostenida → escribir un **ADR-0005** que reemplace al 0004 (principio arc42 §9: si la decisión cambia, se escribe un nuevo ADR como "reemplazado").
- Extraer únicamente el módulo de la tienda dominante como servicio, reutilizando en backend la interfaz aislada por `tiendaId`.

5. **Nota de coherencia:** la alternativa C (servicio por tienda para las 5) sí contradice RES-02/RES-03; el escenario del 40 % es **una sola tienda**, no multiplicar toda la infraestructura → el costo operativo sigue siendo aceptable.

5.3 ¿Cuál fue el resultado final de las pruebas y la medición que dejaron "pendiente de revisión del equipo" en `docs/ia.md` del 06/09?

Respuesta:

La entrada del 06/09 registró como pendiente de revisión del equipo: `npm test` (13/13), `node scripts/medir-aislamiento.js` (salida 0), `node src/corte-vertical.js` y la revisión documental antes de etiquetar. **Ese pendiente quedó resuelto**, con evidencia en el repositorio:

- **npm test: 13/13 pruebas en verde**, incluidas las 4 nuevas de `tests/aislamiento.test.js`.
- **Medición: 0 accesos cruzados en 300 intentos** (arc42 §11.4, ADR-0004 §Trazabilidad, `entregable-corte1.md` §4) → cumple el umbral de ESC-02 con `exit 0`.
- **Corte vertical ejecutado** con salidas capturadas (catálogo → pedidos → pagos → entrega → notificaciones).
- **Etiqueta corte-1** creada sobre el commit `50b92f8`, con CI verde **anterior a la etiqueta** (run `34064927441`).
- **Documentos revisados:** `correcciones.md` (S5) documenta el cierre de cada casilla del corte.

Lo que sigue pendiente (declarado, no ocultado):

- El **PDF de 2 páginas** se sube a Moodle (no se versiona en el repositorio).
- **Activación de SonarCloud** (org/projectKey reales + `SONAR_TOKEN`) para que el análisis estático corra en vivo.

Si preguntan "¿cómo se verifica que no es inventado?" → La medición es reproducible: `node scripts/medir-aislamiento.js` en el estado `812d227` da 2/2 y en `corte-1` da 0/300, con la misma herramienta. Además, la CI quedó en verde antes de la etiqueta.

6. Límites que reconocer (si los preguntan, admitirlos y enmarcarlos)

Límite actual	Postura
Persistencia en memoria (Map)	Correcta y suficiente para el corte y la medición; no es producción. Ruta: PostgreSQL (planeado, contenedor C4). El ADR-0001 ya lo prevé; no cambia las interfaces.
SonarCloud sin activar	Configurado (Dockerfile, sonar-project.properties , job en CI) pero sin token/org. Declarado como pendiente. La CI de pruebas corre y está verde.
Sin capa HTTP	El corte vertical es dominio puro; la interfaz ya acarrea tiendaId para que las rutas futuras no rompan el aislamiento. Corte 2.
Sin telemetría de tráfico por tienda	El disparador del 40 % no es detectable con métricas propias hoy; se habilita con los contenedores planeados (corte 2).
Notificaciones simuladas en memoria	No es push real (A-03); se documenta como simulación en el módulo.

7. Reglas de oro para el turno de preguntas

1. **Nunca inventar una medición.** Si no la tenés, decí "no lo medimos; es un límite declarado".
2. **Toda pregunta se remite a un artefacto:** ADR, escenario (ESC-xx), prueba ([tests/](#)), arc42 §11 o [docs/ia.md](#).
3. **Los 4 responden:** no hay "dueño" de un tema; si el conductor queda bloqueado, otro entra.
4. **Nombrar siempre las restricciones:** RES-02 (equipo de 4) y RES-03 (un semestre) son la justificación de fondo para el monolito modular y para descartar la opción C total.
5. **Pregunta fuera de alcance** → "es un contenedor planeado para el corte 2; decidimos no adelantarlo por RES-02/RES-03".